TECHNICAL PROJECT DOCUMENTATION

SQL Query Generator from Plain English

Natural-language-to-SQL using Groq LLM and BM25-based Retrieval-Augmented Generation

REACT • FASTAPI • GROQ • BM25 RAG • SQLALCHEMY • SUPABASE

Prepared from the project repository implementation

Version 1.0 | 02 November 2026

Repository: github.com/hackwithprem/SQL-Query-Generator

Document Guide

This document describes the project purpose, application architecture, Groq integration, BM25 retrieval process, database context handling, API interfaces, setup, deployment, validation, and operational considerations. It is written for project review, development handover, and future maintenance.

Section	What it explains
1. Project overview	Problem, objectives, scope, and capabilities
2. Technology and architecture	Frontend, backend, AI, data, and request flow
3. Groq LLM integration	API key configuration, provider selection, and prompts
4. BM25 RAG implementation	Knowledge base, tokenization, BM25 scoring, retrieval, and prompt injection
5. SQL generation lifecycle	Schema context, validation, execution, explanation, correction, and history
6. API reference	Main backend endpoint groups
7. Configuration and setup	Environment variables and local run steps
8. Deployment	Vercel, Render, Supabase, and production notes
9. Testing and troubleshooting	Test cases, expected behaviour, and common failures
10. Limitations and future work	Known design boundaries and planned improvements

Terminology The project uses Groq as the LLM API provider. “Grok” is a different product name; the environment variable and Python SDK used here are GROQ_API_KEY and groq.

1. Project Overview

1.1 Project title

SQL Query Generator from Plain English

1.2 Abstract

SQL Query Generator from Plain English is a full-stack web application that converts a user’s natural-language request into a SQL query. The React frontend collects the question and selected data source. A FastAPI backend prepares database-schema context, retrieves relevant SQL examples through a lightweight BM25-based Retrieval-Augmented Generation (RAG) component, and sends the combined instructions to a Large Language Model (LLM) through the Groq API. The generated SQL is checked by a dedicated validator before it can be executed. Users can also inspect query results, request an explanation, correct a failed query, upload tabular data, connect supported external databases, and review query history.

1.3 Problem statement

Writing SQL requires knowledge of SQL syntax, database table structures, joins, filtering, grouping, and aggregation. Users who understand the information they need may still find it difficult to express that request as a correct SQL statement. The project reduces this barrier by accepting plain-English questions and generating SQL that can be reviewed and, where a compatible data source is selected, executed.

1.4 Objectives

- Convert plain-English questions into SQL statements using an LLM.
- Use the actual schema when an uploaded dataset or connected database is selected.
- Improve query patterns by retrieving similar curated SQL examples with BM25 RAG.
- Keep the Groq API credential on the backend instead of exposing it in browser code.
- Validate generated SQL and reject unsafe or multiple-statement requests.
- Support query execution, explanation, correction, data upload, database connections, authentication, and history.

1.5 Main features

Feature	Description
Natural-language SQL generation	Converts a user question into a SQL query using the configured LLM provider.
BM25-based RAG	Retrieves up to three relevant items from a curated library of 45 SQL examples (default settings).
Dynamic schema context	Uses schema information from an uploaded CSV/Excel dataset or a connected database.
Query execution	Executes validated SQL against the selected dataset, sample database, or configured connection.
SQL explanation	Uses the LLM provider to explain the query in accessible language.
SQL correction	Uses the reported database error and schema context to request a corrected query.
Authentication and user data	Provides signup/login and user-scoped dataset and history operations.
Data upload and preview	Supports CSV and Excel upload, metadata extraction, preview, rename, and deletion.
Cloud database connections	Stores connection details and uses a connector to inspect schema and execute queries.
Query history	Records generated/executed queries, execution status, row counts, timings, and errors where available.

2. Technology Stack and Architecture

Layer	Technology	Responsibility
Frontend	React 18, Vite 5, Tailwind CSS, Axios, lucide-react	User interface, forms, result presentation, authentication state, and HTTP requests.
Backend API	Python, FastAPI, Uvicorn, Pydantic	REST endpoints, request validation, orchestration, and API responses.
LLM integration	Groq Python SDK; configured model openai/gpt-oss-120b	Natural-language SQL generation, explanation, and correction.
Retrieval	Pure-Python BM25 / Okapi BM25	Searches the curated SQL-example knowledge base and supplies relevant patterns to the LLM.

Layer	Technology	Responsibility
Database access	SQLAlchemy, sqlparse, database drivers	Schema inspection, SQL validation support, and query execution.
Authentication	JWT, bcrypt; optional Supabase Auth	User authentication and protected operations.
Data services	Pandas, openpyxl, Supabase Storage integration	CSV/Excel parsing and optional cloud file persistence.
Deployment	Vercel frontend; Render backend	Hosts the React application and FastAPI service.

2.1 High-level architecture

Component	Role in the system
React frontend	Captures question, selected dialect/data source, and user actions. Calls backend endpoints through Axios.
FastAPI routes	Receives requests and delegates work to service classes.
SQLGeneratorService	Chooses schema context, retrieves examples, calls the provider, validates output, and records generation history.
RAGService / BM25Index	Indexes 45 curated examples in memory and ranks relevant examples for a natural-language question.
GroqProvider	Builds the system/user messages and calls Groq chat completions.
SchemaService / connectors	Builds prompt context from the selected uploaded dataset or external database.
SQLValidator / SQLExecutor	Checks SQL safety and executes the accepted query against the selected target.
Database / Supabase services	Store application records, user metadata, connection records, history, and optional uploaded files.

2.2 SQL generation request flow

Step	Process
1	User submits a plain-English question
2	React sends POST /api/generate-sql
3	Backend resolves the selected dataset or connection
4	SchemaService builds schema context
5	BM25 retrieves relevant SQL examples
6	Groq receives the question, schema, and examples
7	LLM returns a SQL candidate
8	Validator checks statement count and prohibited operations
9	Backend returns SQL and metadata; history is recorded

Execution is a separate action: the frontend submits the SQL to POST /api/execute-sql. The backend validates and executes it, then returns rows and execution metadata. Generating SQL does not automatically mean that the query has been executed.

2.3 Repository layout (important modules)

Path	Purpose
frontend/	React UI and Axios API client.
backend/app/main.py	Creates the FastAPI application, startup seeding, CORS, and route registration.
backend/app/core/config.py	Central environment-based application configuration.
backend/app/api/routes/	Route modules for authentication, generation, execution, data, schema, history, and other operations.
backend/app/services/sql_generator.py	Coordinates schema discovery, RAG retrieval, LLM call, validation, and history.
backend/app/services/llm_provider.py	Groq, Gemini, and offline mock provider implementations.
backend/app/services/rag/knowledge_base.py	Curated SQL example records (45 examples in the inspected repository).
backend/app/services/rag/rag_service.py	Tokenization, BM25 index construction, retrieval, and prompt formatting.
backend/app/services/sql_validator.py	Read-only and single-statement validation logic.
backend/app/services/sql_executor.py	Database query execution service.
backend/app/services/schema_service.py	Schema inspection and prompt-context construction.
backend/app/database/	SQLAlchemy models, connection/session setup, and seed logic.

3. Groq LLM API Integration

3.1 Why Groq is used

Groq provides the hosted LLM inference API called by the backend. In this application, the LLM performs the language-understanding and SQL synthesis step: it interprets the request, considers schema constraints and retrieved SQL patterns, and produces a candidate SQL statement. The Groq API is not the database and does not execute the generated SQL; execution is handled separately by the backend database layer.

3.2 Current provider configuration

Setting	Current code value / behaviour
LLM_PROVIDER	Defaults to groq. Supports groq, gemini, or mock selection.
GROQ_MODEL	Defaults in config.py to openai/gpt-oss-120b.
GROQ_API_KEY	Secret key read from the backend environment.
SDK call	Groq chat.completions.create(...)
Generation temperature	0.1 in the current Groq SQL-generation call.
Generation max_tokens	600 in the current Groq SQL-generation call.
Provider fallback	If the chosen hosted provider is not configured, get_llm_provider() returns MockOfflineProvider.

3.3 Obtaining and configuring the API key

1. Open the Groq developer console at https://console.groq.com/ and sign in to the account used for the project.
2. Create an API key from the API keys section. Copy it once and keep it private.
3. Open the backend environment file (commonly backend/.env for local development). Do not put the key in a React file or any variable prefixed with VITE_.
4. Add the key and provider/model settings using the example below. Replace the placeholder locally; never commit the actual secret.

```
LLM_PROVIDER=groq
GROQ_API_KEY=gsk_REPLACE_WITH_YOUR_REAL_KEY
GROQ_MODEL=openai/gpt-oss-120b
RAG_ENABLED=true
RAG_TOP_K=3
RAG_MIN_SCORE=0.1
```

5. Restart the backend after changing environment variables so the settings are reloaded.

Secret handling The real API key is intentionally not included in this document. Do not paste it into GitHub, frontend source code, screenshots, issue trackers, or public chat. If a key has been exposed, revoke it in the Groq console and generate a replacement.

3.4 How the backend uses the key

The settings class in backend/app/core/config.py reads GROQ_API_KEY from the process environment or supported .env file. The provider factory in backend/app/services/llm_provider.py checks LLM_PROVIDER and whether a non-placeholder key is present. When configured, GroqProvider initializes the Groq SDK client. The client sends a system message containing SQL safety rules, SQL dialect, schema instructions, and optional retrieved examples, followed by the user question.

3.5 Prompt construction and model output

When a real schema is available, the current Groq prompt instructs the model to use only the tables, columns, and relationships supplied by the schema. When no schema is available, it asks the model to use reasonable conventional names and makes clear that the names are assumptions. In both cases the prompt says to return raw SQL without markdown fences or commentary and to produce only read-only SQL. The backend also strips markdown code fences defensively before validation.

3.6 Provider fallback behaviour

The code includes GeminiProvider and MockOfflineProvider as alternatives. If Groq is selected but the key/client is unavailable, the provider factory falls through to MockOfflineProvider. Therefore, a successful response alone does not prove that Groq was used. During testing, inspect the response provider field or backend logs and verify that the returned provider identifies Groq with the configured model.

Important for demos The offline Mock provider uses rule-based patterns; it is not an LLM. Mention the active provider accurately during a project review.

4. BM25 Retrieval-Augmented Generation (RAG)

4.1 What RAG contributes

Retrieval-Augmented Generation improves the context available to the language model by finding relevant reference material before generation. In this project, the reference material is a curated collection of natural-language questions paired with SQL examples. The RAG module searches these examples for a user question and passes the best matches into the Groq prompt as few-shot guidance.

4.2 RAG approach used in this project

The current runtime implementation is a lightweight, pure-Python BM25 retrieval system. It does not require neural embeddings, a SentenceTransformer model, PyTorch, or a vector database for this retrieval path. It uses lexical term matching and BM25 relevance scores over the question text and example category.

RAG aspect	Project implementation
Knowledge source	45 manually curated SQL examples stored in knowledge_base.py.
Indexed text	Each example question combined with its category name.
Retriever	Custom BM25Index implementation using BM25Okapi-style scoring.
Tokenization	Lowercase, alphanumeric extraction, stop-word filtering, and lightweight suffix normalization.
Index storage	In-memory Python data structures, built once per process and cached on RAGService.
Default top K	3 retrieved examples.
Default minimum score	0.1 (RAG_MIN_SCORE).
Prompt use	Retrieved examples are formatted and injected into the Groq/Gemini SQL-generation prompt.
Failure behaviour	Disabled, empty, no-match, or failed retrieval produces an empty example list; generation continues.

4.3 Knowledge base composition

The inspected knowledge base contains 45 records across SQL patterns and common application domains. Categories include SELECT and DISTINCT, WHERE filtering, IN/BETWEEN/NULL/LIKE, ordering and limits, COUNT/AVG/SUM/MIN/MAX, GROUP BY and HAVING, joins, subqueries, common table expressions (CTEs), CASE expressions, string/date conditions, university marks, sales, and HR/payroll examples.

4.4 Tokenization and normalization

The tokenize() function first converts text to lowercase and extracts alphanumeric terms using a regular expression. It removes common English stop words and ignores short non-numeric tokens. For selected suffix patterns it adds a normalized form as an additional token. For example, common plural or verb endings are normalized using a small rule-based stem_token() helper. This is not a full linguistic stemmer; it is a lightweight vocabulary-alignment step.

4.5 BM25 index construction

When the first retrieval request occurs, RAGService builds the BM25 index from SQL_EXAMPLES and caches it in a class-level variable. For each example, the index stores tokenized text, term frequencies, document length, document frequency, and inverse document frequency. The average document length is calculated across the indexed examples.

4.6 BM25 scoring

For each query term that appears in an example, the implementation adds a BM25 relevance contribution. The score rewards term overlap while adjusting for term frequency and document length. The code uses k1 = 1.5 and b = 0.75. Its inverse document frequency calculation is:

```
IDF(t) = ln( (N - df(t) + 0.5) / (df(t) + 0.5) + 1 )
```

The term contribution follows the BM25 form:

```
score(D, Q) = sum over query terms t of:
 IDF(t) * [ tf(t,D) * (k1 + 1) ]
          / [ tf(t,D) + k1 * (1 - b + b * |D| / avgdl) ]
```

Here, N is the number of examples, df(t) is the number of examples containing term t, tf(t,D) is the term frequency in one example, |D| is that example's token length, and avgdl is the average example length. The implementation sorts examples by score in descending order, filters by the minimum score, and returns at most top_k matches.

4.7 Retrieval and prompt injection flow

1. SQLGeneratorService receives the natural-language question.
2. It calls RAGService.retrieve_relevant_examples(question) when RAG is enabled.
3. RAGService tokenizes the question and searches the cached BM25 index.
4. The retriever returns up to RAG_TOP_K examples that meet RAG_MIN_SCORE.
5. RAGService.format_examples_for_prompt() formats each question, SQL sample, category, and relevance score.
6. The provider adds those references to the system prompt before sending the question to the LLM.
7. The prompt explicitly warns that examples are syntax/structure guidance only; actual table and column names must come from the selected schema.

4.8 Why this is not vector RAG

Vector RAG normally converts text into dense numerical embeddings and compares vectors, often with cosine or another vector similarity measure. The current RAGService does not generate embeddings and does not connect to a vector store. It uses BM25 lexical retrieval. The `similarity` field returned in each retrieved item is populated from the BM25 score for compatibility; it should not be interpreted as a cosine similarity score.

4.9 RAG configuration

Environment variable	Default in code	Meaning
RAG_ENABLED	true	Enables or disables retrieval.
RAG_TOP_K	3	Maximum number of examples returned.

Environment variable	Default in code	Meaning
RAG_MIN_SCORE	0.1	Minimum BM25 score required to include an example.
RAG_SIMILARITY_THRESHOLD	0.1	Backward-compatibility alias used if RAG_MIN_SCORE is absent.

4.10 RAG limitations

- BM25 is lexical: it may miss conceptually similar questions that use very different vocabulary.
- The knowledge base is curated in source code; adding or editing examples requires a code change and deployment.
- The index is process-local and in-memory; each backend process builds its own cached index.
- The retrieval score is not a calibrated probability of correctness.
- Retrieved SQL can contain table/column names that do not exist in a target database. The prompt explicitly says to use the active schema, but the result must still be validated and reviewed.
- RAG helps with reusable SQL patterns; it does not replace schema inspection, database validation, or execution testing.

5. Database Context and SQL Processing

5.1 Schema context modes

Selected source	Current backend behaviour
Uploaded dataset	The generator uses the uploaded dataset's generated table name and column metadata to build a prompt context. The effective dialect is SQLite for this path.
Cloud database connection	The backend decrypts the stored credential, creates the connector, reads the remote schema, and builds prompt context using the connector dialect.
No explicit source	The generator uses schema-free mode. It asks the model to assume conventional table/column names and sets schema_assumed=true. Generated SQL may not execute against the default University Sample DB unless the tables match.

Schema-free generation A query generated without a database schema is a draft based on naming assumptions. To execute it successfully, select a compatible uploaded dataset or connect to a database containing the referenced tables.

5.2 SQL validation

After the provider returns SQL, SQLGeneratorService sends it to SQLValidator.validate(). The validator parses the SQL with sqlparse, rejects empty input and multiple executable statements, checks the first meaningful token, and scans parsed tokens for prohibited operations. Its forbidden-keyword set includes data modification and schema-changing operations such as INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, REPLACE, GRANT, REVOKE, PRAGMA, ATTACH, DETACH, EXECUTE, and related commands. It also blocks SELECT INTO patterns and formats accepted SQL.

The validator is a defensive application layer, not a substitute for database permissions. Production database credentials should be restricted to the minimum privileges required, ideally read-only for query execution.

5.3 SQL execution

Execution is exposed through a separate endpoint. SQLExecutor executes the submitted SQL against the selected dataset, local database, or connector and returns an execution response. Application configuration defines MAX_RESULT_ROWS = 500 and QUERY_TIMEOUT_SECONDS = 10. The frontend API client currently uses an Axios request timeout of 30 seconds. These values should be reviewed together when query sizes, hosting tiers, or database latency change.

5.4 Explanation and correction

The explanation service selects the relevant schema context and asks the configured provider to describe the SQL in simple language, including tables, filters, joins, grouping, and ordering where applicable. The correction service passes the original question, failed SQL, error message, and schema context to the provider and requests a corrected query plus an explanation. Correction output should be validated again before execution.

5.5 History

The project records generation and execution activity through HistoryService. Depending on the operation, records can include natural-language question, SQL text, executed status, status label, error information, execution duration, result row count, and user identifiers. History endpoints allow the frontend to retrieve and clear the current user's history.

6. Backend API Reference

The following routes were confirmed in the current frontend API client and backend route modules. The API base path in the local frontend is configured through VITE_API_BASE_URL; the routes below are relative to the backend host.

Method	Endpoint	Purpose / notes
GET	/	Service overview and links to docs/health.
GET	/api/health	Backend health status.
GET	/api/schema	Returns schema information for the configured application database.
POST	/api/generate-sql	Generates SQL from a natural-language question; may include dataset_id or connection_id.
POST	/api/execute-sql	Validates and executes SQL against selected target.
POST	/api/explain-sql	Returns a natural-language explanation for SQL.
POST	/api/correct-sql	Attempts to repair SQL using original question, failed SQL, error, and schema context.

Method	Endpoint	Purpose / notes
POST	/api/auth/signup	Creates an account and returns an authentication response.
POST	/api/auth/login	Authenticates a user.
POST	/api/auth/logout	Signs out and clears the access cookie/client token.
GET	/api/auth/me	Returns the current user profile.
PUT	/api/auth/profile	Updates the current user display name.
POST	/api/data/upload	Uploads CSV/Excel data (authenticated).
GET	/api/data/files	Lists the current user's uploaded datasets.
GET	/api/data/files/{file_id}/preview	Returns a dataset preview.
PATCH	/api/data/files/{file_id}/rename	Renames a dataset.
DELETE	/api/data/files/{file_id}	Deletes a dataset.
GET	/api/history	Retrieves query history.
POST	/api/history/clear	Clears query history.
POST	/api/connections/test	Tests external database connection details.
POST	/api/connections	Creates a saved external database connection.
GET	/api/connections	Lists saved external database connections.
GET	/api/connections/{connection_id}/schema	Returns schema for a saved connection.
DELETE	/api/connections/{connection_id}	Deletes a saved connection.

FastAPI interactive API documentation is exposed at /docs when the backend is running. Use that page to inspect request/response schemas and test endpoints in the current deployment.

7. Configuration and Local Setup

7.1 Environment variables

Variable	Purpose / example
APP_ENV	development locally; production on hosted backend.
LLM_PROVIDER	groq (recommended current configuration).
GROQ_API_KEY	Private Groq API key; backend only.
GROQ_MODEL	openai/gpt-oss-120b (current config default).
RAG_ENABLED	true or false.
RAG_TOP_K	3 by default.
RAG_MIN_SCORE	0.1 by default.
DATABASE_URL	SQLAlchemy connection URL; code defaults to a local SQLite database.

Variable	Purpose / example
CORS_ORIGINS	Allowed frontend origins; use valid JSON array or comma-separated values per config parser.
JWT_SECRET_KEY	Strong unique secret for signing tokens; replace the development default.
SUPABASE_URL / SUPABASE_KEY	Optional Supabase project configuration.
SUPABASE_SERVICE_ROLE_KEY	Optional server-only privileged key; never expose in frontend.
SUPABASE_STORAGE_BUCKET	Private dataset storage bucket name; default is private-datasets.
MAX_RESULT_ROWS / QUERY_TIMEOUT_SECONDS	Configured row and query-time limits.

7.2 Local backend setup

```
cd backend
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS/Linux
source .venv/bin/activate
pip install -r requirements.txt
# Create backend/.env and configure GROQ_API_KEY and other values
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

7.3 Local frontend setup

```
cd frontend
npm install
# Create frontend/.env.local
VITE_API_BASE_URL=http://127.0.0.1:8000/api
npm run dev
```

Open the Vite development URL shown in the terminal (typically http://localhost:5173). The FastAPI server is typically available at http://127.0.0.1:8000 and interactive documentation at http://127.0.0.1:8000/docs.

7.4 Environment file rules

- Keep backend secrets in backend/.env or the hosting provider's secret environment settings.
- Keep frontend configuration limited to public values such as the API base URL. VITE_ variables are bundled into browser code and are not secret.
- Ensure .env files are ignored by Git and inspect git status before committing.
- Do not commit real Groq, Supabase, JWT, or database credentials.
- If credentials were previously committed or shared publicly, rotate/revoke them and replace the deployed values.

8. Deployment and Operations

8.1 Current known deployment targets

Component	Platform	Known project URL
Frontend	Vercel	https://sql-query-generator-delta.vercel.app/
Backend	Render	https://sql-query-generator-api.onrender.com/
Repository	GitHub	https://github.com/hackwithprem/SQL-Query-Generator

These are the deployment URLs recorded in the project handover context. Confirm them in the Vercel and Render dashboards if the project has since been renamed or redeployed.

8.2 Render backend configuration

Setting	Value
Root directory	backend
Build command	pip install -r requirements.txt
Start command	uvicorn app.main:app --host 0.0.0.0 --port $PORT
Environment secrets	GROQ_API_KEY, JWT_SECRET_KEY, and any required Supabase/database settings.
CORS_ORIGINS	Include the exact deployed Vercel origin, for example ["https://sql-query-generator-delta.vercel.app"].
RAG settings	RAG_ENABLED=true; RAG_TOP_K=3; RAG_MIN_SCORE=0.1, unless intentionally disabled.

8.3 Vercel frontend configuration

Setting	Value
Root directory	frontend
Framework/build	Vite; npm run build; output directory dist.
Environment variable	VITE_API_BASE_URL=https://sql-query-generator-api.onrender.com/api

8.4 Hosting considerations

- On a free Render instance, inactivity spin-down can create a cold start on the first request after a period without traffic.
- Low-memory instances can restart when the application exceeds available RAM. The pure-Python BM25 path avoids loading large embedding models, but normal application and database memory still matter.
- Local SQLite files on an ephemeral hosting filesystem may not provide durable persistence across redeploys or instance replacement. Use managed persistent storage/database services for production data.
- Use Supabase Storage or another persistent object store for uploaded file durability when configured.

- Keep CORS origins restricted to the actual frontend origins in production; do not leave wildcard CORS enabled for production deployments.

9. Testing and Verification

9.1 Core functional tests

Test	Example input/action	Expected check
Groq provider	Generate SQL with GROQ_API_KEY configured.	Response provider identifies Groq and the configured model.
Provider fallback	Temporarily test without a valid Groq key in a safe local environment.	Provider falls back to MockOfflineProvider; do not mistake it for Groq output.
BM25 retrieval	Ask: “Find products priced higher than 100 dollars.”	Related filtering example may be retrieved; `rag_applied` true when at least one example is returned.
RAG disabled	Set RAG_ENABLED=false and restart backend.	Generation continues without retrieved examples; `rag_applied` false.
Schema-aware generation	Select a dataset or connected database and ask for an existing table/column.	Prompt contains selected schema; query uses schema-defined names.
Schema-free generation	Generate a question without selecting a data source.	Response indicates schema was assumed; query may need a compatible database.
Read-only validation	Try an UPDATE or DROP statement through execution.	Validator rejects prohibited statements.
Multi-statement validation	Submit SELECT ...; DROP TABLE ...	Validator rejects multiple statements.
Query execution	Generate and execute a valid SELECT against a compatible dataset.	Rows and execution metadata are returned.
Explain/correct	Ask for SQL explanation; then submit a failed query for correction.	Explanation/correction response is returned with context.
Upload access	Upload a CSV/Excel file while logged in.	Dataset record and preview metadata are returned for that user.

9.2 Example questions for a demo

- List all students with their department names.
- Show students who scored more than 80 marks.
- Find the average score for each course.
- Count the number of students in each course.
- Show the top five students ordered by GPA.
- Find products priced higher than 100 dollars.
- List customers with orders placed between two dates.

For reliable execution, choose a data source whose actual schema contains the requested tables and columns. If no source is selected, generated table/column names are assumptions rather than verified database objects.

9.3 How to verify that RAG was applied

1. Confirm RAG_ENABLED=true in the backend environment and restart/redeploy the service.
2. Send a natural-language question related to one or more curated examples.
3. Inspect the generate-sql response. In the current SQL generator, rag_applied is set from whether the retrieved-example list is non-empty; retrieved_examples may also be present.
4. If rag_applied is false, check the exact setting, question vocabulary, minimum score, and whether any knowledge-base terms match. A question can be valid even when no example clears the score threshold.

10. Troubleshooting

Symptom	Likely cause	Action
Provider says Mock/Offline	Missing/invalid GROQ_API_KEY, wrong LLM_PROVIDER, or client initialization issue.	Check Render/backend environment variables and logs; verify key is backend-only; restart service.
Groq authentication error	Key revoked, malformed, or copied with whitespace.	Create a fresh key in Groq Console; update backend secret and redeploy.
RAG not applied	RAG_ENABLED=false, no BM25 term matches, or score below threshold.	Verify RAG settings; test with terminology represented in the curated examples; inspect retrieved_examples.
Slow first request	Render free-tier cold start or upstream model latency.	Check service status/logs; wait for service startup; monitor request latency.
Frontend network error	Incorrect VITE_API_BASE_URL, CORS mismatch, or backend unavailable.	Verify Vercel environment URL, Render health endpoint, and CORS_ORIGINS.
Generated SQL fails on execution	Schema-free query, incorrect assumed names, or SQL dialect mismatch.	Select an uploaded dataset/connection; inspect schema; use correction and review generated SQL.
Dataset disappears after redeploy	File or SQLite data stored on ephemeral filesystem.	Use persistent managed storage/database and verify Supabase Storage configuration.
Request timeout	Backend cold start, slow LLM response, or query exceeds frontend timeout.	Inspect backend logs; check Axios timeout (currently 30 seconds) and align timeout policies carefully.
CORS blocked	Frontend origin missing from CORS_ORIGINS.	Add exact origin in Render environment as a valid list and redeploy.

11. Security and Responsible Use

- Keep GROQ_API_KEY and all database/Supabase privileged keys on the backend only.
- Use a strong production JWT_SECRET_KEY; do not keep a default/example secret in a deployed environment.
- Use least-privilege database accounts. Prefer read-only credentials for query execution.
- Validate SQL on the backend even when the frontend displays only generated queries.
- Review generated SQL before execution, especially if generated without a real schema.
- Enforce authentication and user ownership checks for uploaded datasets and database connection records.

- Use HTTPS in production and configure secure cookies and deployment CORS according to the production domain.
- Avoid logging secrets or full connection passwords. Protect logs and user-uploaded datasets.
- Apply upload size and row limits; verify file type and parsing behaviour before accepting user files.

Security note from code review The inspected config.py contains a development fallback JWT secret and the auth routes set the cookie `secure=False`. Set a strong JWT_SECRET_KEY in the host environment and review cookie security settings before treating the deployment as production-grade.

12. Limitations and Future Enhancements

12.1 Current limitations

- BM25 retrieval depends on lexical overlap and may not find paraphrases with little shared vocabulary.
- The SQL-example knowledge base is static source data rather than a user-managed or continuously updated corpus.
- Schema-free generation can produce SQL that cannot run against the sample database or the selected target.
- The LLM can still generate syntactically plausible but semantically incorrect SQL; validation does not prove query correctness.
- Retrieval scores should be treated as ranking signals, not confidence probabilities.
- Free-tier hosting can introduce cold starts, resource limits, and storage persistence constraints.

12.2 Future enhancement ideas

- Expand and version the SQL knowledge base with reviewed, dialect-specific examples.
- Add retrieval evaluation using a test set with expected relevant examples and recall/precision measurements.
- Consider a hybrid retriever (BM25 plus embeddings) if paraphrase-heavy queries become a measured weakness.
- Add query-plan inspection, stricter execution timeouts, and database-level read-only enforcement.
- Introduce structured logging, request IDs, latency monitoring, and explicit provider/RAG diagnostics.
- Add persistent database and object storage for production deployments.
- Add automated tests for exact row limits, selected columns, sorting, joins, SQL injection attempts, and dialect handling.
- Provide a schema-aware SQL review panel showing the tables/columns used before execution.

13. Key Technical Concepts for Project Review

Question	Suggested explanation
What is the role of Groq?	Groq hosts the LLM inference API. The backend sends the natural-language question, schema context, and retrieved SQL examples to the model to produce a candidate SQL query.
What is RAG?	Retrieval-Augmented Generation adds relevant reference information to the model prompt before generation. Here, references are curated natural-language-to-SQL examples.
Why BM25?	BM25 is a lightweight text retrieval algorithm. It ranks examples using term frequency, inverse document frequency, and document-length normalization without requiring a neural embedding model.
Is this vector RAG?	No. The current request path uses pure-Python BM25 lexical retrieval. It does not use vector embeddings or a vector database.
How many RAG examples are present?	The inspected knowledge_base.py contains 45 curated example records.
What is top K?	The maximum number of retrieved examples to add to the prompt. The current default is three.
What happens if RAG fails?	The RAG service returns an empty list and SQL generation continues. RAG is an enhancement, not a hard dependency.
Why pass database schema to the LLM?	It constrains generation to actual table and column names when a dataset or database connection is selected.
Why validate SQL after generation?	LLM output is not automatically trusted. The validator rejects multiple statements and dangerous operations before execution.
Does generating a query execute it?	No. Generation and execution use separate backend endpoints and separate user actions.

14. Project References

The following source files are the primary implementation references for the features described in this document. Open the repository to review the latest code because implementation details may change between commits.

Topic	Repository file
Configuration and environment	backend/app/core/config.py
Groq, Gemini, and mock providers	backend/app/services/llm_provider.py
SQL generation orchestration	backend/app/services/sql_generator.py
BM25 retrieval implementation	backend/app/services/rag/rag_service.py
Curated SQL knowledge base	backend/app/services/rag/knowledge_base.py
SQL validation	backend/app/services/sql_validator.py
FastAPI app and route registration	backend/app/main.py
Generation / execution routes	backend/app/api/routes/generate.py and execute.py
Data upload routes	backend/app/api/routes/data.py
Frontend API integration	frontend/src/api/client.js
Python dependencies	backend/requirements.txt
Frontend dependencies	frontend/package.json

Repository: https://github.com/hackwithprem/SQL-Query-Generator

Conclusion

The project combines a React-based interface, a FastAPI backend, a Groq-hosted language model, and a lightweight BM25 RAG component to generate SQL from natural language. Its key technical workflow is the combination of schema context and retrieved SQL patterns before LLM generation, followed by backend validation and optional execution. The strongest project-review explanation is that BM25 improves the prompt with relevant examples, Groq generates the SQL candidate, and the backend—not the LLM—controls validation and database execution.